

Hackathon FIAP X — Fase 5 — Entrega no portal do aluno (PDF)

Grupo: Oficina Turbo (106)
Aluno: Felipe Ricarte Magalhães

1. Dois repositórios de microsserviços (Fase 5 — FIAP X)

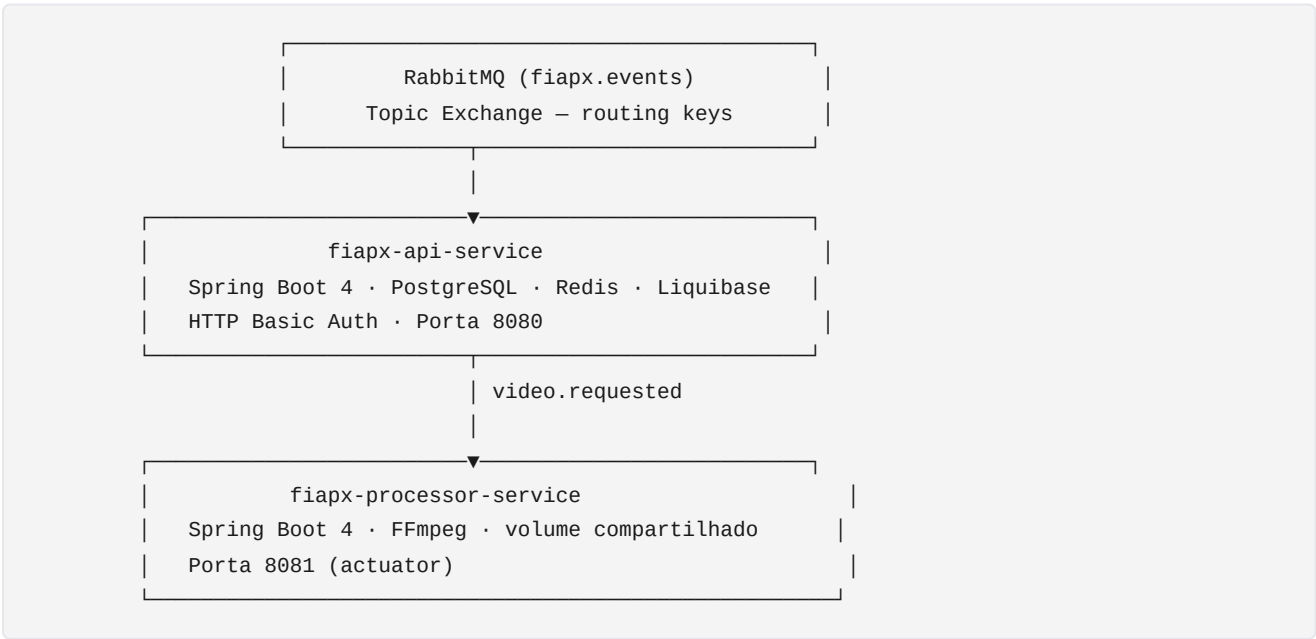
#	Repositório	Responsabilidade	Porta
1	https://github.com/ricartefelipe/fiapx-api-service	API REST — autenticação, upload, listagem de status, download do ZIP	8080
2	https://github.com/ricartefelipe/fiapx-processor-service	Worker assíncrono — extração de frames com FFmpeg e geração do ZIP	8081

Branches: `main` (estável, CI verde), `develop` (integração).
Acesso ao avaliador SOAT: usuário `soat-architecture` convidado com leitura nos dois repositórios.

Documentação técnica no repositório central:
<https://github.com/ricartefelipe/fiapx-api-service/tree/main/docs/fase5>

2. Arquitetura — Microsserviços e fluxo de processamento

2.1 Visão geral



Infraestrutura compartilhada (Docker Compose): PostgreSQL, RabbitMQ, Redis, Prometheus e Grafana sobem junto com os dois microserviços.

2.2 Fluxo upload → fila → FFmpeg → ZIP → download

Etapa	Componente	Ação
1	Cliente	POST /api/videos (multipart) com HTTP Basic Auth
2	API Service	Persiste VideoJob (status QUEUED), salva arquivo e publica video.requested
3	RabbitMQ	Entrega mensagem na fila video.processing
4	Processor Service	Consume evento, extrai frames com FFmpeg, gera ZIP no volume compartilhado
5	Processor Service	Publica video.completed ou video.failed
6	API Service	Atualiza status do job; em erro, registra notificação via log
7	Cliente	GET /api/videos/{id} para acompanhar status; GET /api/videos/{id}/download quando COMPLETED

2.3 Eventos RabbitMQ

Exchange	Routing Key	Produtor	Consumidor
fiapx.events	video.requested	API	Processor
fiapx.events	video.completed	Processor	API
fiapx.events	video.failed	Processor	API

3. Stack técnica

Componente	Tecnologia
Linguagem	Java 21
Framework	Spring Boot 4.0.6
API REST	Spring Web + Spring Security (HTTP Basic Auth)
Mensageria	RabbitMQ + Spring AMQP
Banco relacional	PostgreSQL + Liquibase
Cache	Redis

Componente	Tecnologia
Processamento de vídeo	FFmpeg (via ProcessBuilder no processor)
Documentação API	SpringDoc OpenAPI / Swagger UI
Testes	JUnit 5 + Mockito + Testcontainers
Cobertura	JaCoCo
CI/CD	GitHub Actions
Containers	Docker + Docker Compose
Observabilidade	Micrometer + Prometheus + Grafana

4. Requisitos do hackathon — checklist

Requisito	Status	Evidência
Processar mais de um vídeo simultaneamente	✓	Fila RabbitMQ + worker assíncrono; múltiplos jobs em paralelo
Não perder requisições em picos de carga	✓	Filas duráveis no RabbitMQ; API retorna 202 Accepted
Proteção por usuário e senha	✓	HTTP Basic Auth (fiapx / fiapx123 demo)
Listagem de status dos vídeos por usuário	✓	GET /api/videos e GET /api/videos/{id}
Notificação em caso de erro	✓	LogNotificationService + evento video.failed
Persistência de dados	✓	PostgreSQL com entidades users e video_jobs
Arquitetura escalável	✓	API e processor desacoplados; fila absorve picos
Versionamento no GitHub	✓	Dois repositórios públicos com GitFlow
Testes automatizados	✓	./mvnw -Pci clean verify em ambos os serviços
CI/CD	✓	GitHub Actions com build, testes e push Docker (GHCR)
Docker Compose funcional	✓	docker compose up -d --build
Teste E2E	✓	./scripts/e2e-test.sh (upload → COMPLETED → download ZIP)

5. Cobertura de testes e CI

5.1 JaCoCo (local, perfil `ci`)

Serviço	Cobertura de instruções	Testes
fiapx-api-service	60,0%	Unitários (controller, service, security) + <code>@SpringBootTest</code>
fiapx-processor-service	53,8%	Unitários (listener, ZIP, processing) + <code>@SpringBootTest</code>

Cobertura medida com `./mvnw -Pci clean verify`. Camadas de integração (RabbitMQ listeners, FFmpeg real) concentram código ainda não coberto por testes unitários.

5.2 CI/CD — GitHub Actions

Repositório	Workflow	Status em <code>main</code>
fiapx-api-service	<code>ci.yml</code> + <code>enforce-gitflow.yml</code>	✅ Verde
fiapx-processor-service	<code>ci.yml</code> + <code>enforce-gitflow.yml</code>	✅ Verde

Evidências:

- <https://github.com/ricartefelipe/fiapx-api-service/actions>
- <https://github.com/ricartefelipe/fiapx-processor-service/actions>

Imagens Docker publicadas em: `ghcr.io/ricartefelipe/fiapx-api-service` e `ghcr.io/ricartefelipe/fiapx-processor-service`

5.3 SonarCloud

Item	Status
Propriedades no <code>pom.xml</code>	✅ Configurado (<code>ricartefelipe_fiapx-api-service</code> , <code>ricartefelipe_fiapx-processor-service</code>)
Análise automática no CI	N/A — token SonarCloud ainda não integrado ao pipeline
Dashboards	N/A — aguardando primeira análise publicada

6. Como executar

6.1 Pré-requisitos

- Docker e Docker Compose
- Repositórios `fiapx-api-service` e `fiapx-processor-service` no mesmo diretório pai
- Opcional: FFmpeg no host (para o script E2E gerar vídeo de teste)

6.2 Stack completa (recomendado)

```
cd fiapx-api-service
docker compose up -d --build
```

Serviço	URL / Porta
API	http://localhost:8080/api
Swagger UI	http://localhost:8080/api/swagger-ui.html
Processor (health)	http://localhost:8081/actuator/health
RabbitMQ Management	http://localhost:15672 (guest/guest)
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin/admin)

Credenciais da API: `fiapx` / `fiapx123`

6.3 Teste E2E automatizado

```
cd fiapx-api-service
chmod +x scripts/e2e-test.sh
./scripts/e2e-test.sh
```

O script aguarda a API, envia upload, poll até `COMPLETED` e valida o download do ZIP.

6.4 Testes unitários

```
./mvnw -Pci clean verify # em cada repositório
```

7. GitFlow aplicado

Fluxo documentado em `docs/GITFLOW.md` (ambos os repositórios):

Branch	Uso
feature/*	Novas funcionalidades → PR para develop
fix/*	Correções → PR para develop
release/*	Preparar versão → PR develop → main
hotfix/*	Correção urgente → PR para main + sync develop

O workflow `enforce-gitflow.yml` **bloqueia** PR de `feature/*` direto para `main`.

Release entregue: PR #4 (`develop` → `main`) mergeado com CI verde em 24/07/2026.

8. Observabilidade

Ferramenta	Acesso
Prometheus	http://localhost:9090
Grafana	http://localhost:3000
Métricas	<code>/actuator/prometheus</code> em API (8080) e Processor (8081)

9. Vídeo demonstrativo (≤ 10 minutos)

Status: PENDENTE — o aluno envia o link separadamente no portal.

Link: *(inserir URL YouTube/Vimeo antes do envio no portal)*

Roteiro sugerido:

- `docker compose up -d --build` — stack completa no ar
- Swagger UI — autenticação Basic Auth
- Upload de vídeo → job `QUEUED` → `PROCESSING` → `COMPLETED`
- RabbitMQ Management — fila e exchange `fiapx.events`
- Download do ZIP com frames extraídos
- `./scripts/e2e-test.sh` — validação automatizada
- GitHub Actions — CI verde nos dois repositórios
- Prometheus/Grafana — métricas dos serviços

10. Pacote zipado (portal)

Arquivo `fase5-microservicos.zip` enviado no portal contém:

- Código-fonte de `fiapx-api-service` (sem `target/`, sem `.git/`)
- Código-fonte de `fiapx-processor-service` (sem `target/`, sem `.git/`)
- Este PDF (`entrega-portal-fase5.pdf`)

Arquivo **fase5.zip** contém a pasta `fase5/` com o PDF e o ZIP de código.

Entrega Fase 5 — Hackathon FIAP X — plataforma de processamento de vídeos em microsserviços com RabbitMQ, PostgreSQL e Docker Compose.